

CS260 Review Topics & Study Guide

- ☒ 7 layer model - make flashcards
- ☒ Ethernet
- ☒ IPv4 & IPv6
- ☐ Data Grams vs Streams
- ☐ Frames, packets
- ☐ IP & Mac Addressing - learn bitmasks & calculations
- ☒ WAN & NAT
- ☐ DHCP & DNS
- ☐ Routing & Gateways
- ☐ HTTP & Postel's Law

Ethernet:

Each node has locally unique ID & a MAC Address

Mac address is 48 bits long, in hex

- First half is manufacturer ID
- Second half "unique" val set by manufacturer, burned into hardware

IP Addresses:

32 bits, 4 8bit dotted quads from 0 to 255

Subnet Mask: Networks within networks or "subnets"

- Need to know subnet mask to know if devices are on same network
- Defines size of network & relationship
/24 = First 24 bits are masked with 1

IP = 192.168.1.100, mask = 255.255.255.0

IP AND mask = 192.168.1.0

IP AND NOT mask = 0.0.0.100

App 7
Present 6
Session 5
Transport 4
Network 3
Link 2
Physical 1

2-Link layer - local comms.

- physical connections, wires, radio signal, relatively small

3- Network Layer

- logical connections

Routers & Gateways:

Router

- Connects 2 networks

- Makes dec. about which traffic goes where

Gateway

- Connects 2 different types of networks together

What if we want to send non local packets? Send to the default gateway

- Where firewalls & security is placed

- IPv4 addresses routable, so they send to router, which sends to other routers

- Routers use 'routing tables' which typically in TCP uses the "Border Gateway Protocol", each route has a cost (think dijkstra)

Classful Networks: bit patterns of addresses determine scope

- Class A: 8 bits - Class B: 14 bits (2 reserved) Class C: 21 bits (3 reserved)

- lasted until 92 till CIDR & classless networks



Network Number $\rightarrow$ routing ID = Network Number / Subnet Mask Bits

Example 192.168.0.100/24, network # = 24

WAN & NAT Review:

NAT - Network Address Translation

Made because IPv4 public addresses are limited

Nat translates each device's private address to its public address of the router

WAN as opposed to LAN, wide area network

DHCP: Dynamic Host Configuration Protocol

- Automatically assigns IP addresses & other info
- Simplifies & abstracts address assignment, can use a pool

Client: DHCPDISCOVER → Server: DHCPOFFER → Client: DHCPREQUEST
- ip addr., sub. mask,

Server: DHCPACK

UDP Seq. of Calls:

1. Init the library - `WSAStartup()`
2. Make socket → `Socket()` → `htonl()` → `inet_pton()`
3. If sending msg:
 - Make network addr. obj for dest
 - Send data to destin. addr via socket `sendto()`
4. If receiving
 1. Make network addr obj for the source
 2. Bind socket to src addr.
 3. Wait for incoming msg (`recvfrom()`)